

珞樱 V2 开发计划稿

一、背景说明

珞樱 V2 架构已经完成第一阶段设计与编码，系统已初步具备新的分层结构与扩展能力。接下来的工作重点，不再是继续无节制堆叠功能，而是先验证 V2 在新环境中的可运行性、可维护性与可扩展性，确保系统在换环境、换平台、换接入方式后依然能够稳定运行。

当前阶段的总体目标可以概括为三句话：

- 先活下来
- 再让大家看懂
- 最后开始拆能力、做前端、走向多平台

也就是说，近期最重要的不是“功能看起来很多”，而是“V2 真能跑、真能维护、真能继续发展”。

二、阶段总目标

1. 首要目标：完成 V2 基础可运行验证

首先，将 V2 仓库 clone 到服务器，完成依赖安装、配置整理与基础启动测试。

这一阶段暂时不以 LLbot 或 QQ 连接成功作为硬性标准。即使 QQ 接不上、LLbot 连接失败，也不影响本阶段验收。现阶段的核心任务是确认：

- V2 在服务器环境中能否正常安装与启动
- 当前依赖是否齐全、版本是否兼容
- 核心配置是否存在强环境耦合
- 主程序启动后是否存在致命报错
- skill 注册、agent 调用链、日志机制是否正常
- 换系统后架构是否会直接崩溃

换句话说，这一阶段是“生存验证阶段”。

重点不是上线，而是确认 V2 不会因为环境变化而直接炸掉。

2. 第二目标：组织核心成员学习 V2 架构

在完成基础运行验证后，将组织核心成员集中学习 V2 架构，并在下一次会议中推进整体讲解。

学习 V2 的目的，不是要求所有人马上理解全部代码细节，而是让核心成员建立统一认识，明确以下问题：

- V2 与旧版本相比，核心变化是什么
- agent、skill、transport 三层分别负责什么
- 新功能应该接在哪一层
- 哪些地方可以安全扩展，哪些地方不能随意修改
- 为什么未来不能长期只依赖 QQ

目标是让 V2 从“只有少数人能维护的系统”，逐步变成“团队可协作维护的系统”。

3. 第三目标：启动服务化拆分试点

在基础运行与架构学习完成后，将启动部分模块的服务化拆分试点，验证 V2 从“单体智能体”向“主 agent + 外部服务 + 子 agent”方向演进的可行性。

当前优先考虑的两个模块为：

- 天气模块：拆为独立服务，并由子 agent 调用
- 论文模块：拆为独立服务，并由子 agent 调用

选择这两个模块作为第一批拆分对象，主要原因是：

- 功能边界较清晰，容易定义输入输出
- 与主流程耦合相对有限，适合试点
- 能够验证服务调用、异常处理、模块复用等关键机制
- 对未来更多模块拆分具有示范意义

4. 第四目标：推动 Web 前端形成最小可用版本

V2 不能长期依赖 QQ 作为唯一交互入口，Web 前端必须尽快落地，至少形成一个基础可用的独立交互端。

前端近期目标不是做完整产品，也不是做精细视觉，而是尽快打通最小闭环，使用户可以在不依赖 QQ 的情况下完成：

- 登录

- 发起对话
- 查看回答
- 查看历史记录
- 完成基础使用流程

只有 Web 前端开始可用，V2 才真正具备脱离单平台约束、走向更广泛使用场景的可能。

三、阶段划分与里程碑

阶段一：部署验证阶段

阶段目标

将 V2 成功部署到服务器，完成基础启动与运行验证。

核心任务

- 将 V2 仓库 clone 到目标服务器
- 安装运行所需依赖
- 检查 Python 版本、环境变量、系统包依赖
- 整理配置文件与默认配置项
- 进行首次启动测试
- 检查日志输出是否正常
- 检查 skill 是否能被成功注册
- 检查 agent 主流程是否能在脱离 QQ 的情况下基本运行
- 记录环境相关问题与缺失项

验收标准

满足以下条件则视为阶段完成：

- 仓库成功 clone
- 项目依赖安装完成

- 主程序可启动
- 启动过程中无致命报错
- skill 注册机制正常
- 日志输出正常
- 至少能完成一次本地 mock 调用、控制台调用或非 QQ 环境调试调用
- 形成初步部署记录与问题清单

预期产出

- 一份可复现的服务器部署流程
- 一份已知环境问题记录
- 一份依赖与配置核对结果
- 一版“V2 已可运行”的结论说明

阶段二：QQ 接入恢复与替代调试阶段

阶段目标

在不阻塞主线的前提下，尝试恢复 QQ 接入；若短期不稳定，则建立非 QQ 的替代调试入口。

核心任务

- 安装 LLbot 并测试基础环境
- 检查 V2 transport 与 QQ 平台接入逻辑
- 验证消息接收、消息转换、消息发送链路
- 检查 reply、图片、上下文等消息结构传递情况
- 若 QQ 接入不稳定，则优先构建 mock transport 或 Web transport 调试通道
- 避免整个系统测试完全依赖 QQ 平台

验收标准

满足以下任一类即可判定阶段可接受：

A. QQ 接入成功：

- 能成功连接 QQ
- 能接收消息
- 能触发 agent 处理
- 能完成基础回复

B. QQ 接入暂未成功，但替代调试路径建立：

- 可通过 mock / Web / 控制台方式进入主处理链
- 核心逻辑不因 QQ 不可用而无法测试
- transport 层问题被局部隔离，不阻塞整体开发

预期产出

- 一份 QQ 接入调试记录
- 一份 transport 调试说明
- 一个不依赖 QQ 的临时或长期调试入口

阶段三：架构学习与团队协作阶段

阶段目标

使核心成员理解 V2 的基本分层、调用流程和扩展方式，降低项目维护门槛。

核心任务

- 整理 V2 架构讲解材料
- 说明 agent / skill / transport 的职责边界
- 展示一次完整调用流程
- 讲解如何新增一个 skill
- 讲解如何新增一个 transport
- 明确哪些修改属于正常扩展，哪些属于破坏性更新
- 建立后续开发协作约定

会议建议内容

建议下次会议围绕以下内容展开：

1. V2 总体结构讲解
2. 核心目录说明
3. skill 编写流程说明
4. transport 适配思路说明
5. 服务化演进方向说明
6. Web 前端的定位与职责说明
7. 开发协作边界与评审机制说明

验收标准

- 核心成员对系统分层有基本共同认识
- 至少部分成员可以独立编写简单 skill
- 至少部分成员可以定位 transport 入口
- 团队内部对“什么可以改、什么不能乱改”形成共识

预期产出

- 一份架构讲解文档或会议纪要
- 一份模块职责说明
- 一份团队协作边界说明

阶段四：服务化拆分试点阶段

阶段目标

验证将部分能力从主系统中拆出，改造成独立服务并由子 agent 调用的可行性。

优先试点模块

- 天气模块
- 论文模块

核心任务

- 重新定义天气模块边界
- 重新定义论文模块边界
- 为两个模块设计独立服务接口
- 设计子 agent 调用方式
- 实现主 agent 到外部服务的调用链
- 增加超时、失败、异常兜底策略
- 对拆分前后行为进行对比测试

本阶段特别关注的问题

- 服务接口是否足够清晰
- 主 agent 是否仍保持简洁
- 外部服务失败是否会拖垮主系统
- 新的子 agent 是否会带来额外复杂度
- 模块拆分是否真的提升可维护性，而不是徒增工程负担

验收标准

- 天气模块能通过服务调用正常完成能力输出
- 论文模块能通过服务调用正常完成能力输出
- 外部服务异常时主流程不会整体崩溃
- 主系统与服务之间接口相对稳定
- 拆分结果具有可复用价值

预期产出

- 天气服务原型
 - 论文服务原型
 - 子 agent 调用样例
 - 一份模块拆分经验总结
-

阶段五：Web 前端最小可用版本阶段

阶段目标

构建一个不依赖 QQ 的最小可用前端，使珞樱具备基础独立入口。

最小实现范围

当前阶段前端不追求大而全，建议以最小闭环为准，至少实现：

- 登录页
- 对话页
- 历史记录页
- 基础个人中心页
- 基础导航
- 请求状态反馈
- 错误提示
- 空状态提示

核心任务

- 明确前后端接口
- 打通登录流程
- 打通发送问题与接收回答流程
- 支持展示流式或完整回答
- 支持查看历史对话
- 实现基础页面跳转
- 做到基础可用，而非只停留在静态页面

验收标准

- 用户可完成登录
- 用户可发起一次对话
- 用户可看到 AI 回答
- 用户可查看历史记录

- 无需依赖 QQ 即可完成完整交互闭环
- 基础页面在 PC 端可正常使用

预期产出

- Web 前端最小可用版本
- 一套基础接口联调结果
- 一份前端待完善项清单

四、团队分工建议

1. 总设计师职责

作为 V2 总设计师，当前主要负责：

- 把控整体架构方向
- 决定核心分层和调用边界
- 审核破坏性更新
- 设计模块服务化路线
- 确定公共接口规范
- 协调不同成员工作避免互相冲突
- 平衡短期交付与长期可维护性

同时，优先亲自推进以下工作：

- 天气模块服务化试点
- 论文模块服务化试点
- **架构讲解**与关键决策说明

2. 其他后端成员职责方向

其他后端成员当前可优先投入以下方向：

- 学习并编写新的 skill
- 阅读并理解 skill 的注册和调用流程

- 学习 transport 的适配机制
- 补充工具类、辅助模块、公共能力
- 完善配置、日志、异常处理
- 编写测试样例或 mock 调试工具

建议优先做“低风险增量开发”，尽量避免直接修改主链路核心逻辑。

3. Web 前端成员职责方向

Web 前端同学是近期非常关键的一环，应尽快推动珞樱形成独立使用入口。

当前建议重点关注：

- 登录页与鉴权流程
- 对话页基本交互
- 历史记录展示
- 页面导航
- 加载态与错误态反馈
- 基础 UI 可用性

前端当前最重要的任务是“做出能用的东西”，而不是先追求高完成度视觉方案。先可用，再优化。

4. 运维 / 部署 / 文档方向职责

如果有成员可投入这一方向，优先级也很高。重点包括：

- 服务器部署流程整理
 - 环境依赖清单维护
 - 配置示例文件整理
 - 启动方式说明
 - 常见报错记录
 - 日志查看方式说明
 - 调试手册编写
-

五、协作边界与修改规范

为了保护 V2 当前架构稳定性，现阶段对不同类型改动作如下区分。

1. 正常开发范围

以下内容属于正常开发范围，成员可在充分理解代码后进行开发与提交：

- 新增 skill
- 修改或优化现有 skill
- 新增 transport
- 优化 transport 适配逻辑
- 前端页面开发
- 新增辅助工具类
- 补充测试
- 补充文档

2. 破坏性更新范围

以下改动属于破坏性更新，必须先由总设计师研判后再实施：

- 核心上下文结构修改
- agent 主调用链修改
- skill 注册机制修改
- transport 抽象层总体结构修改
- 配置系统结构性修改
- 历史记录结构调整
- 消息模型变更
- 核心调度机制修改
- 目录级重构
- 跨模块公共接口改动

3. 协作原则

- 新功能优先以 skill、service 或 transport 的方式扩展

- 非必要不直接侵入主流程
 - 提交前应说明改动范围和影响模块
 - 涉及公共结构时应先评审
 - 每个新模块尽量附带基本说明文档
 - 重要改动需考虑回滚路径
-

六、当前阶段重点风险

1. 环境迁移风险

V2 在本地可运行，不代表在服务器环境中一定可运行。依赖缺失、版本冲突、系统包差异、路径问题都可能导致部署失败。

2. 平台耦合风险

如果所有调试都严重依赖 QQ，那么一旦 QQ 接入不稳定，整个 V2 的研发节奏都会被拖死。因此必须尽快建立非 QQ 的调试入口。

3. 架构认知不统一风险

如果团队成员对 V2 的分层理解不一致，后续很容易出现越层修改、重复造轮子、公共接口混乱等问题。

4. 过早大改核心风险

当前阶段最怕大家一边学架构，一边顺手重构核心流程，最后导致系统长期不稳定。V2 现在需要的是稳住，而不是全面推倒重来。

5. 前端长期缺位风险

如果 Web 前端迟迟不落地，项目就会一直被 QQ 平台绑架，不利于后续展示、推广和扩展。

6. 服务化拆分失控风险

服务化是方向，但不是为了拆而拆。如果接口不清、异常处理不好、观测不到位，拆分反而会增加复杂度。因此必须先做试点，再决定是否扩大范围。

七、近期执行顺序建议

建议近期按照以下顺序推进：

第一顺位

完成 V2 服务器部署与基础运行验证。

第二顺位

同步推进 QQ 接入调试，但不允许其阻塞主流程验证。

第三顺位

准备 V2 架构学习材料，在下一次会议中完成核心成员培训。

第四顺位

启动天气模块、论文模块服务化拆分试点。

第五顺位

推动 Web 前端完成最小可用版本。

这个顺序的核心思路是：

先验证系统活着，再保证团队能接手，再尝试架构演进，最后建立真正独立的产品入口。

八、建议形成的阶段性产物

为了避免做了很多事但没有沉淀，建议这一阶段至少产出以下内容：

- V2 服务器部署文档
- 依赖与配置说明
- 架构讲解材料
- 团队协作边界说明
- QQ / transport 调试记录
- 天气服务拆分文档
- 论文服务拆分文档
- Web 前端最小可用版本说明
- 当前问题清单与下一阶段待办列表

这些文档和产物不是形式主义，而是后续团队扩张、功能迭代和新成员接手的基础。

九、总结

珞樱 V2 已经完成了“写出来”的第一步，接下来真正关键的是完成“跑起来、讲清楚、拆得动、接得广”。

当前阶段的核心判断标准，不是功能数量，而是以下四点：

- V2 能否在**新环境**中稳定运行
- 团队是否**逐步理解**并能**协作维护**
- 架构是否真的支持**服务化和扩展**
- 产品是否能摆脱对**单一平台**的依赖

因此，接下来所有工作都应围绕这四件事展开。

在保证整体架构稳定的前提下，有序推进部署验证、架构学习、模块拆分与前端建设，把 V2 从“新架构代码”真正推进为“可持续发展的系统”。